Multivariable graphics · specifying graphs · axes & labels

Thursday, May 28, 2026

Today: Thursday May 28

Before class

- **Perusall Assignment**: *DC* §8.1–8.3

Plan for today

- Picking up where we left off: facets in depth
- The *Data Computing* "specify a graph" workflow
- Axes, labels, scales, and when to transform them
- Coordinate systems (briefly)
- In-class activity: multivariable plot-matching

Helpful links

- Syllabus · AI policy · Schedule · *R4DS* Ch. 9 · *DC* Ch. 8 · ggplot2 reference

Quick recap from Wednesday

We covered the **vocabulary** (frame, glyph, aesthetic, scale, guide) and the **mechanics** (mapping vs. setting, layered geoms, stats, position adjustments).

Today we extend that to plots with **three, four, or more variables**, and clean up two things we glossed over: the workflow for going from a question to a finished plot, and how to fine-tune axes once you have one.

> **Tip**
>
> The grammar of graphics is the *vocabulary*. The DC "specify a graph" steps are the *grammar* — the order you put the words in.

Today's dataset: penguins

We're going back to the `palmerpenguins` dataset, because:

- It's small (344 rows) and easy to reason about
- It has a mix of numeric, categorical, and grouped variables
- You already know what's in it from week 1
- The relationships between variables let us demo *multi-variable* plots cleanly

```
library(palmerpenguins)
```

```
Attaching package: 'palmerpenguins'

The following objects are masked from 'package:datasets':

    penguins, penguins_raw
```

```
glimpse(penguins)
```

```
Rows: 344
Columns: 8
$ species           <fct> Adelie, Adelie, Adelie, Adelie, Adelie, Adelie, Adel~
$ island            <fct> Torgersen, Torgersen, Torgersen, Torgersen, Torgerse~
$ bill_length_mm    <dbl> 39.1, 39.5, 40.3, NA, 36.7, 39.3, 38.9, 39.2, 34.1, ~
$ bill_depth_mm     <dbl> 18.7, 17.4, 18.0, NA, 19.3, 20.6, 17.8, 19.6, 18.1, ~
$ flipper_length_mm <int> 181, 186, 195, NA, 193, 190, 181, 195, 193, 190, 186~
$ body_mass_g       <int> 3750, 3800, 3250, NA, 3450, 3650, 3625, 4675, 3475, ~
$ sex               <fct> male, female, female, NA, female, male, female, male~
$ year              <int> 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007~
```

Multivariable graphics

How many variables can one plot show?

You already know how to map **2 variables** (x, y) and add a 3rd with color.

Each additional aesthetic lets you encode another variable:

Table 1: Adding variables to a plot.

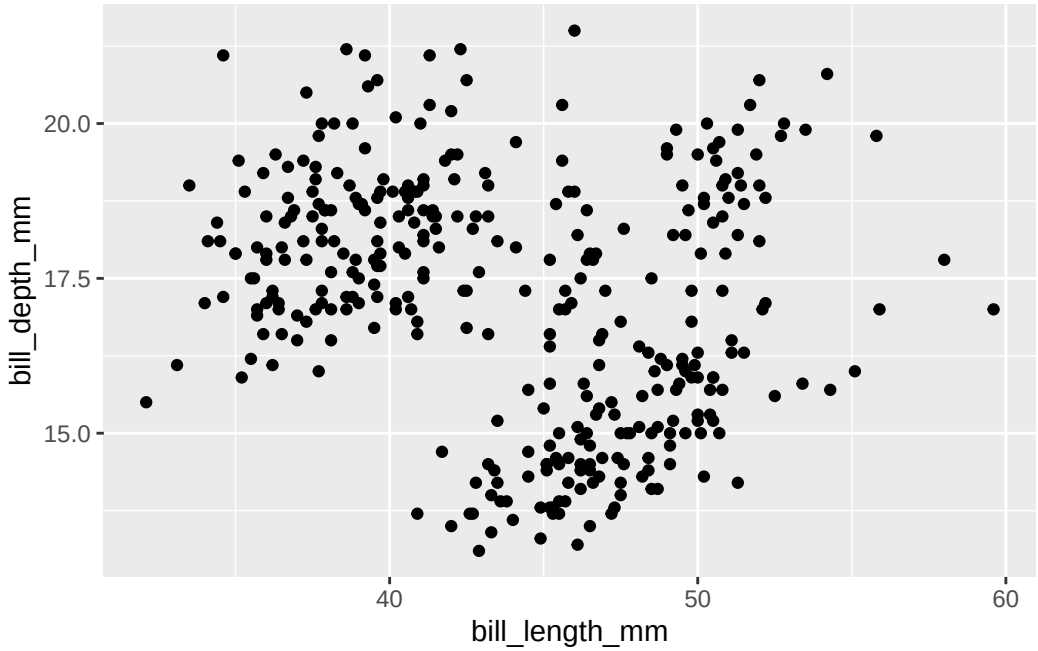
# variables	How
2	x, y
3	+ color OR shape OR size
4	+ color AND (shape OR size)
5	+ facet by a categorical variable
6+	+ facet_grid two variables, or use multiple panels

> **Warning**
>
> Just because you *can* show 6 variables on one plot doesn't mean you *should*. Past 3–4, readers start losing track. Faceting is usually clearer than piling on aesthetics.

Two variables

The starting point. Just x and y.

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point(na.rm = TRUE)
```

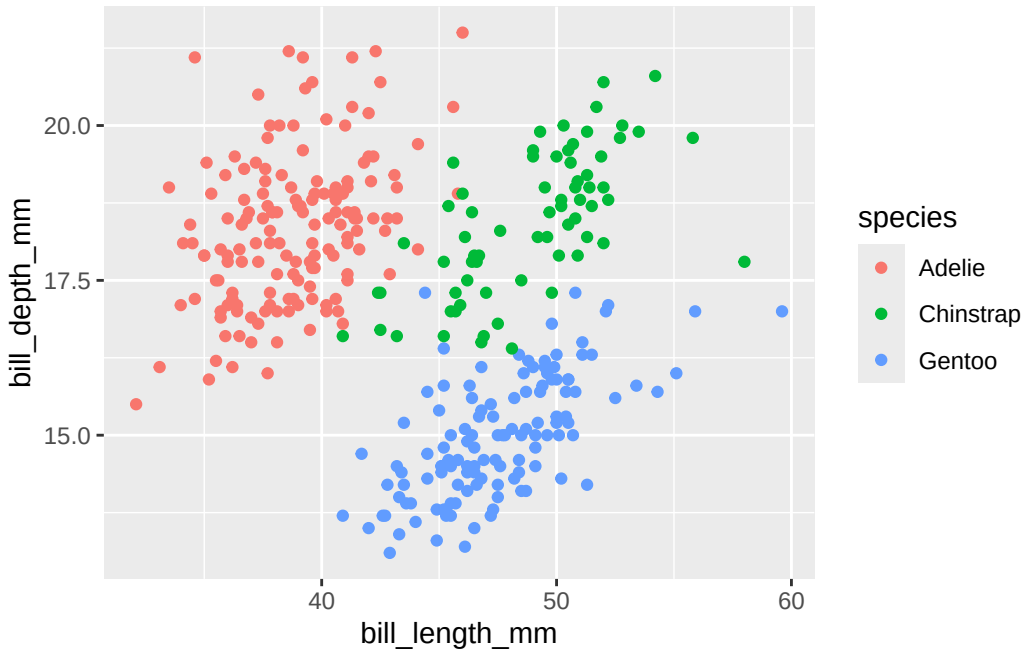


> **Simpson's paradox preview**
>
> This scatter looks like *longer bills are shallower*. We'll see in a minute why that's the opposite of the truth.

Three variables, with color

Add `species` as the third variable via color:

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm, color = species)) +
  geom_point(na.rm = TRUE)
```

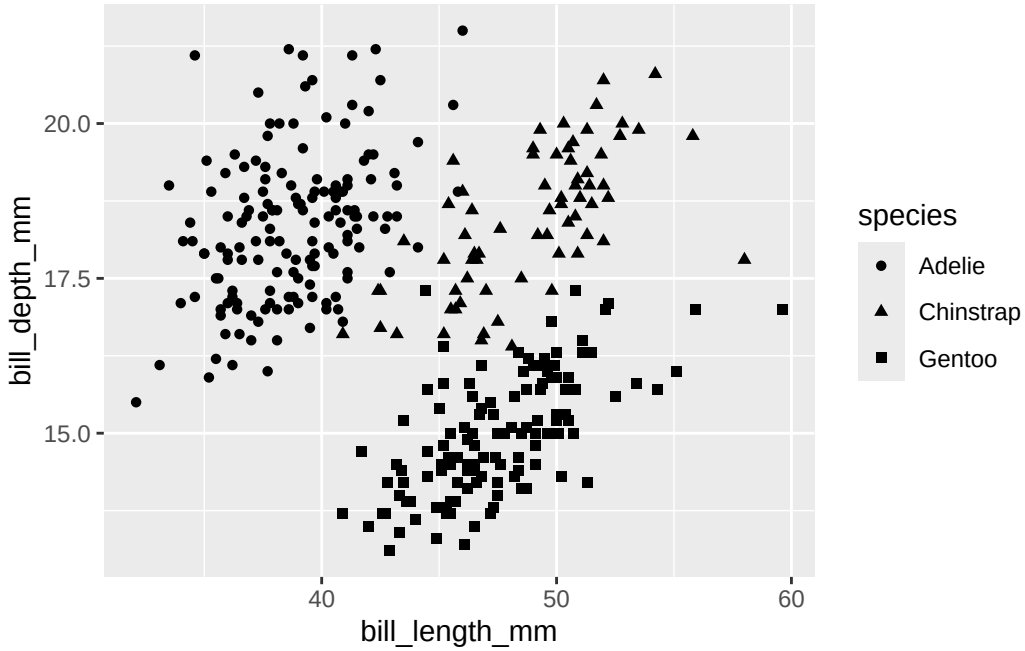


The negative trend we saw before was **Simpson's paradox** — *within* each species, longer bills are deeper. The pooled trend ran the wrong way because Gentoos (long bills, shallow bills) are a separate cluster from the other species.

Three variables, with shape

Same data, but using shape instead of color:

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm, shape = species)) +
  geom_point(na.rm = TRUE)
```

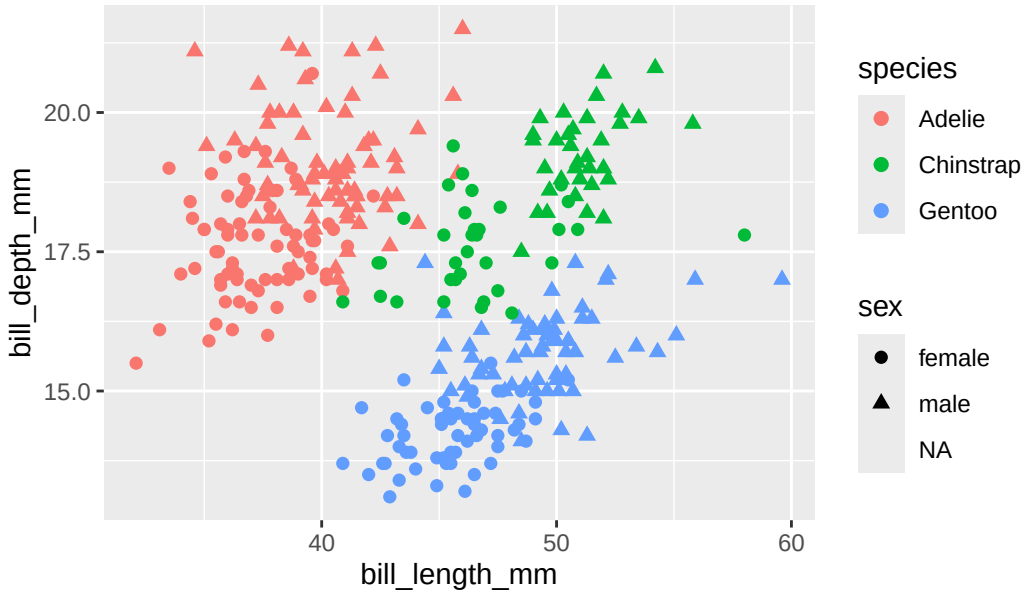


When is shape better than color? When your output might be **printed in black and white**, or when you specifically need color free for *another* variable.

Four variables: combine color and shape

Map two different variables to color and shape at once:

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm,
                     color = species, shape = sex)) +
  geom_point(size = 2, na.rm = TRUE)
```

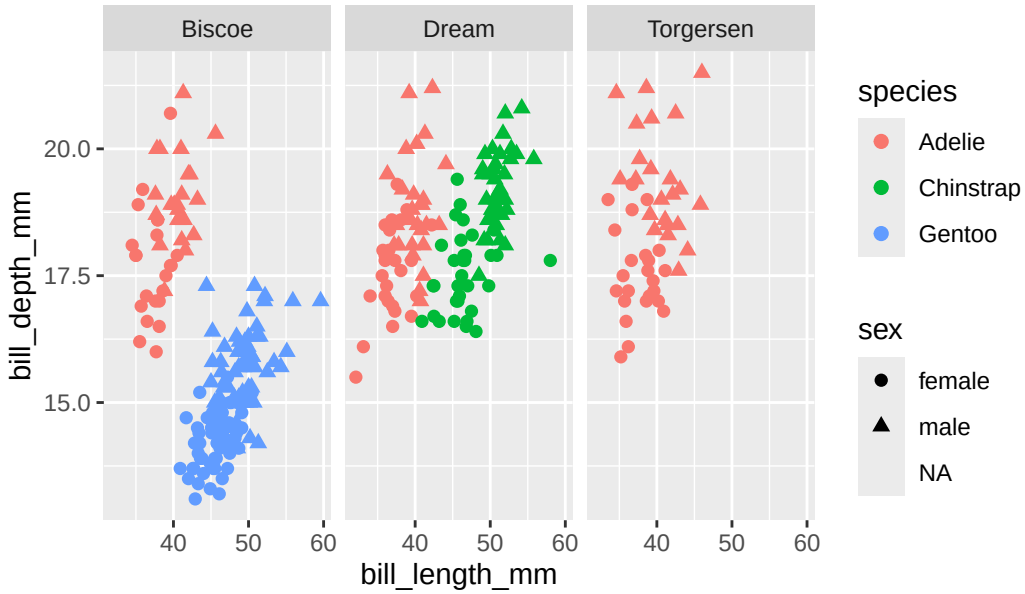


This works — but readers have to track *two* legends and *two* mappings. The visual load gets heavy fast.

Five variables: add a facet

When you're tempted to map a 5th aesthetic, try faceting instead:

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm,
                     color = species, shape = sex)) +
  geom_point(size = 2, na.rm = TRUE) +
  facet_wrap(~island)
```



Now we can see species × sex × island all at once — and crucially, see that Gentoos only live on Biscoe and Adélies live everywhere.

Facets in depth

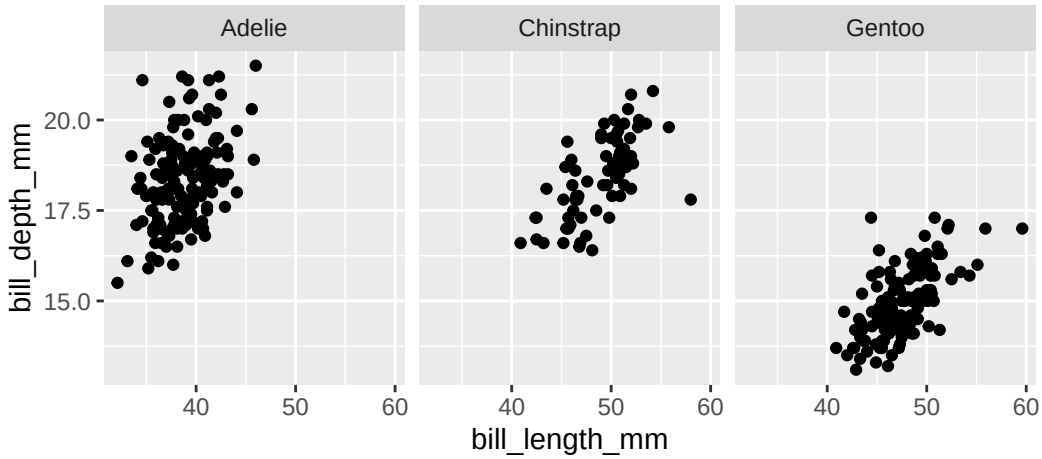
`facet_wrap()` vs `facet_grid()`

Two related functions, different jobs:

`facet_wrap(~ var)`

One variable. Wraps panels into a rectangle. Use when you just want one panel per category.

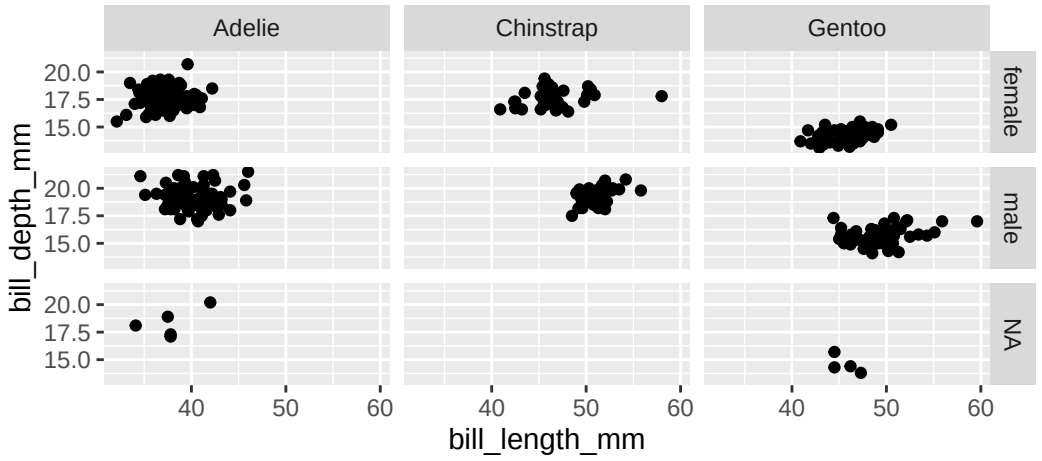
```
ggplot(penguins,
       aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point(na.rm = TRUE) +
  facet_wrap(~species)
```



`facet_grid(rows ~ cols)`

Two variables. Forces a grid layout. Use when you want all combinations crossed.

```
ggplot(penguins,
       aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point(na.rm = TRUE) +
  facet_grid(sex ~ species)
```



The `.` placeholder in `facet_grid()`

Use `.` when you only want to facet on one axis but still want `facet_grid`'s behavior:

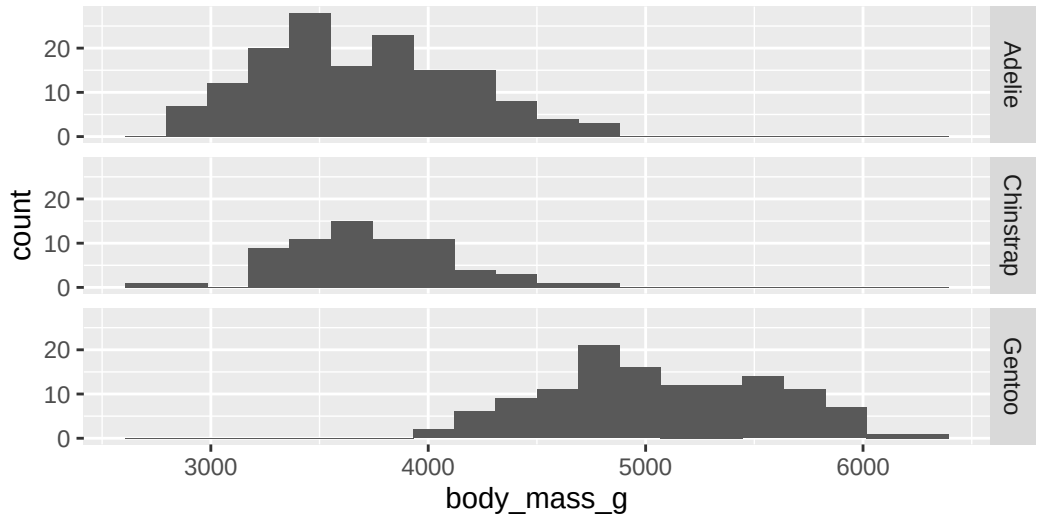
```r
# Stack rows by species:
... + facet_grid(species ~ .)

# Spread columns by species:
... + facet_grid(. ~ species)
```

The first gives you one tall column of three plots stacked vertically; the second gives you one wide row of three plots side-by-side.

When rows beat columns (or vice versa)

Rows make it easy to compare values along the **x-axis** across groups.

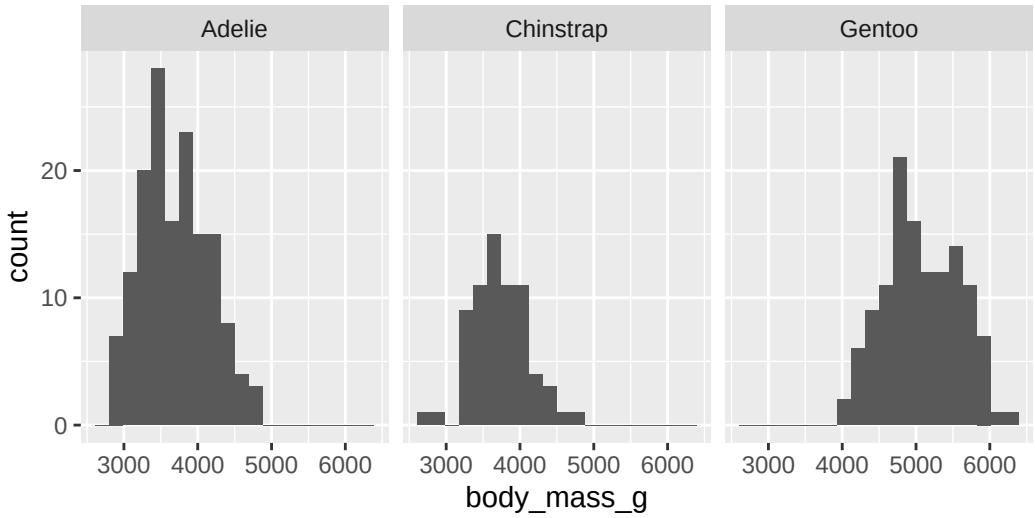
```r
ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram(bins = 20, na.rm = TRUE) +
  facet_grid(species ~ .)
```



Easy to see Gentoos weigh more.

Columns make it easy to compare values along the **y-axis** across groups.

```r
ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram(bins = 20, na.rm = TRUE) +
  facet_grid(. ~ species)
```



Same info, less obvious.

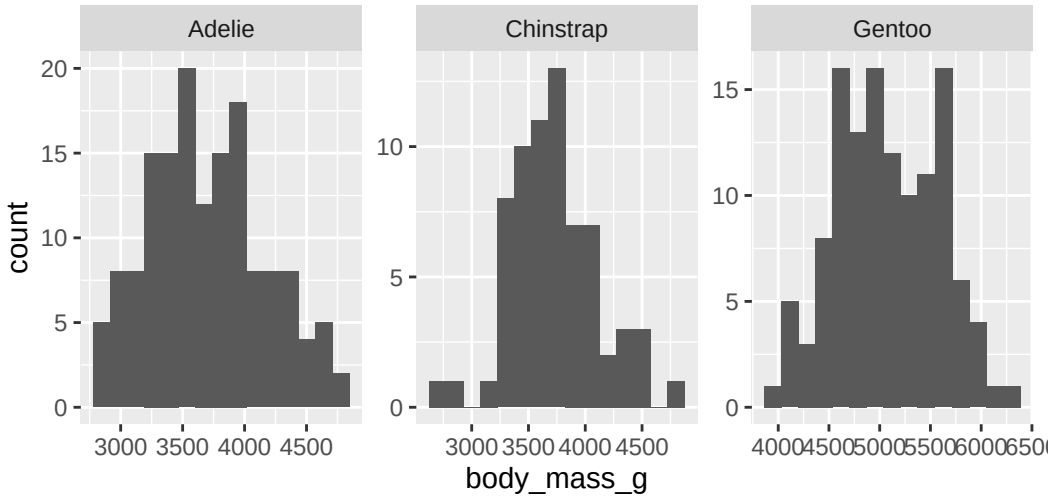
Rule of thumb: facet so that the **comparison you care about** is along the **longer axis** of the panel.

Free vs. fixed scales in facets

By default, every facet shares the same x and y scale. That's almost always what you want for comparison.

But occasionally a facet has a *very different* range, and forcing them to share crushes the detail:

```
ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram(bins = 15, na.rm = TRUE) +
  facet_wrap(~species, scales = "free")
```



- `scales = "free_x"` — let x-axes differ across facets
- `scales = "free_y"` — let y-axes differ across facets
- `scales = "free"` — let both differ

> **Warning**
>
> Free scales make individual facets readable but make *comparison* across facets misleading. If your goal is comparison, leave scales fixed.

When to facet vs. when to use color

A choice you'll face constantly:

Table 2: Color or facet?

Use **color** when…	Use **facets** when…
You want all groups on one set of axes	You want each group's structure visible
The trend is what matters	Each group has its own substructure (clusters, outliers)
You have 2–4 categories	You have 3+ categories (color gets noisy)
You want to highlight a *comparison*	You want to compare *patterns within*

For larger datasets with overlap, **facets almost always win** — color blends together in dense regions.

Specifying a graph

The DC workflow

From DC §8.1, here's the process for building any ggplot:

1. **Define the frame.** Which variables go on x and y?
2. **Describe a graphics layer.**
    - Choose a glyph (geom).
    - Map variables to the aesthetics it supports.
3. **Repeat step 2** for any additional layers.
4. **Refine** with axes, labels, facets, scales.

This mirrors the function structure of ggplot2: `ggplot()` makes the frame, `geom_*()` adds layers, `aes()` does the mapping, `+` adds refinements.

Worked example: bill length vs. depth

Question: How does the relationship between bill length and bill depth vary by species and sex?

Step 1 — Define the frame. x and y are both numeric measurements:

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm))
```

Step 2 — First layer: points. Map `species` to color, `sex` to shape:

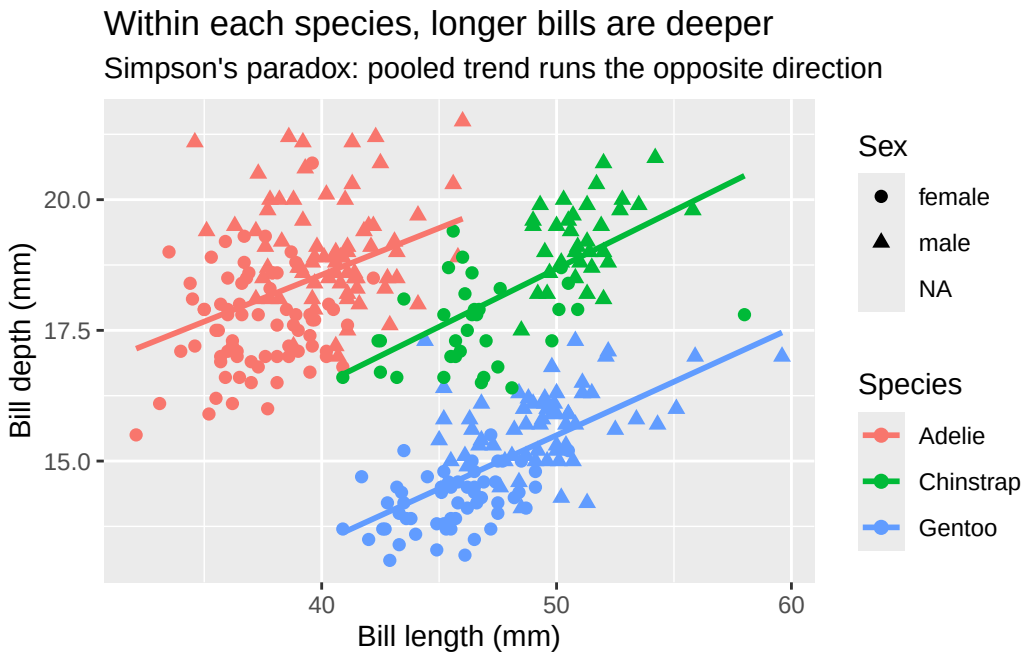
```
... + geom_point(aes(color = species, shape = sex), size = 2, na.rm = TRUE)
```

Step 3 — Second layer: smoothers, one per species. No sex distinction here:

```
... + geom_smooth(aes(color = species), method = "lm", se = FALSE)
```

Putting it all together

```r
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point(aes(color = species, shape = sex), size = 2, na.rm = TRUE) +
  geom_smooth(aes(color = species), method = "lm", se = FALSE, na.rm = TRUE) +
  labs(x = "Bill length (mm)",
       y = "Bill depth (mm)",
       title = "Within each species, longer bills are deeper",
       subtitle = "Simpson's paradox: pooled trend runs the opposite direction",
       color = "Species", shape = "Sex")
```



Axes, labels, and scales

Labels are not optional

The single highest-impact, lowest-effort improvement you can make to any plot:

```r
... + labs(
  x = "Bill length (mm)",
  y = "Bill depth (mm)",
  title = "Within each species, longer bills are deeper",
  subtitle = "Adélies, Chinstraps, and Gentoos sampled in 2007-2009",
  caption = "Source: palmerpenguins package",
```

```
  color = "Species"
)
```

What goes where:

- `title` — what the plot shows
- `subtitle` — the nuance or context the title leaves out
- `x`, `y` — what's on the axis, *with units*
- `caption` — data source, methodology notes
- legend titles — name of the aesthetic mapping (`color =`, `shape =`, etc.)

DC's axis-customization toolkit

From §8.2:

Table 3: Axis tools.

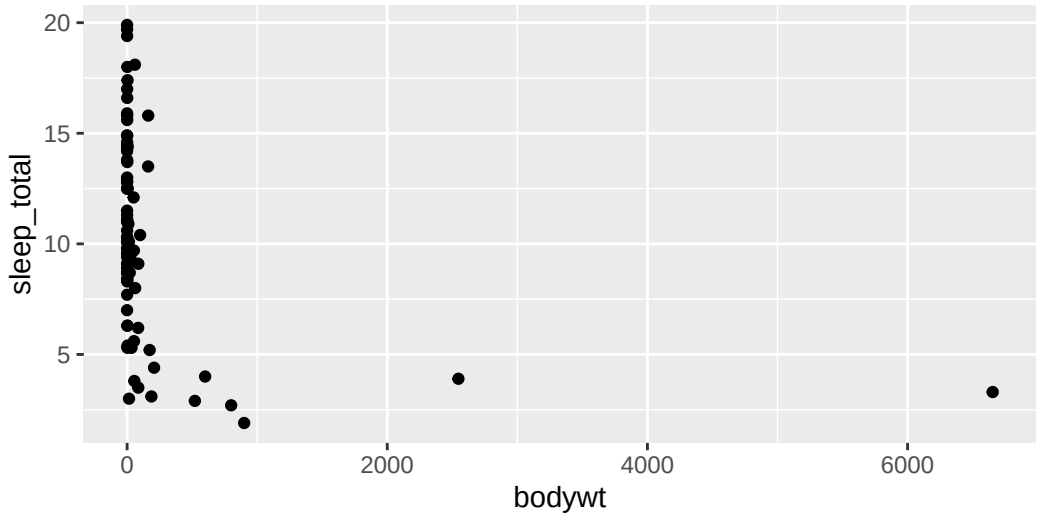
Function	What it does	Example
`xlab()`, `ylab()`	One axis label	`+ ylab("Body mass (g)")`
`xlim()`, `ylim()`	Set axis range	`+ xlim(30, 60)`
`scale_x_log10()`	Log scale on x	`+ scale_x_log10()`
`scale_y_log10()`	Log scale on y	`+ scale_y_log10()`
`coord_cartesian(ylim = c(...))`	Zoom *without* dropping data	safer than `ylim()`

> ⚠ Warning
>
> `ylim(0, 100)` **drops** any row with y outside that range. This affects smoothers, summary statistics, everything. Use `coord_cartesian(ylim = c(0, 100))` to just zoom the visible window.

When to log-transform an axis

If a variable spans **several orders of magnitude**, the raw scale will squish everything together at the bottom:

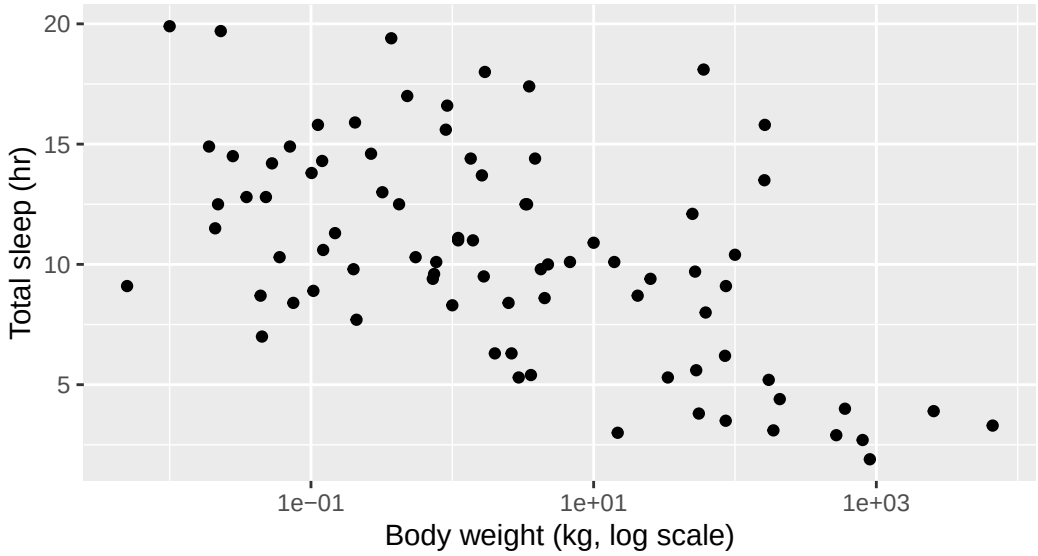
```
ggplot(msleep, aes(x = bodywt, y = sleep_total)) +
  geom_point(na.rm = TRUE)
```



Everything is jammed against the left edge. The elephant ruins the plot.

Log axis fixes it

```
ggplot(msleep, aes(x = bodywt, y = sleep_total)) +
  geom_point(na.rm = TRUE) +
  scale_x_log10() +
  labs(x = "Body weight (kg, log scale)", y = "Total sleep (hr)")
```



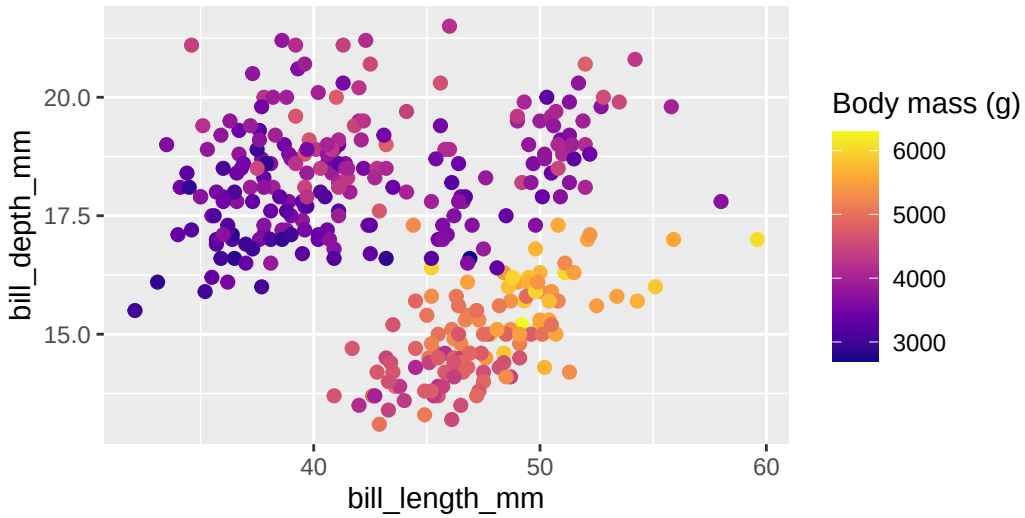
Now you can see the actual relationship: heavier animals sleep less. The slope wasn't visible before because 99% of the data was in the leftmost 5% of the x-axis.

> Tip
>
> **When to log:** ratios matter more than absolute differences (e.g., the difference between 1g and 10g matters about as much as 1kg vs 10kg); your data spans orders of magnitude; you're plotting counts/prices/populations.

Continuous color scales

A categorical variable maps to a *qualitative* palette (distinct hues). A continuous one maps to a *gradient*.

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm,
                     color = body_mass_g)) +
  geom_point(size = 2, na.rm = TRUE) +
  scale_color_viridis_c(option = "plasma") +
  labs(color = "Body mass (g)")
```



`scale_color_viridis_c()` is the gold standard for continuous color: perceptually uniform, colorblind-friendly, prints fine in greyscale. The `_c` is for continuous; `_d` exists for discrete.

Sequential vs. diverging palettes

Sequential — for data that runs from low to high (or none → lots).

`scale_color_viridis_c()`, `scale_color_gradient(low, high)`

Body mass, age, count, distance from origin.

Diverging — for data that runs above and below a meaningful midpoint.

`scale_color_gradient2(low, mid, high, midpoint = 0)`

Temperature anomaly, growth rate, correlation, vote margin.

> ⚠ Warning
>
> The classic mistake is using a diverging palette (red-white-blue) for sequential data, or vice versa. Diverging implies "above/below zero matters" — if your data has no meaningful midpoint, don't use one.

Coordinate systems (briefly)

When you might change the coord

Most plots use the default Cartesian coordinates. A few cases call for something else:

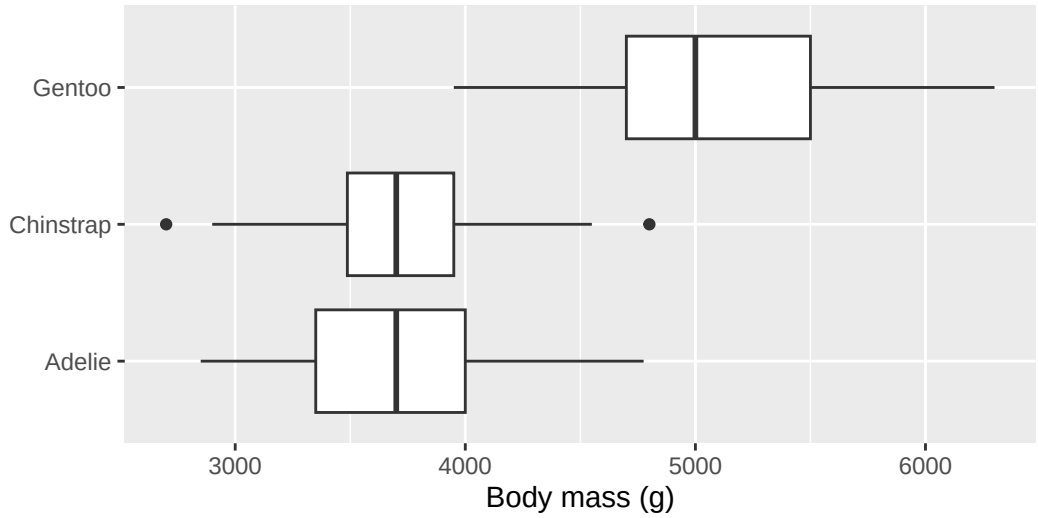
Table 4: Coordinate functions you might actually use.

Coord	When
`coord_cartesian()`	Zoom without dropping data
`coord_flip()`	Swap x and y (handy for long category labels)
`coord_fixed()`	Force 1:1 aspect ratio (when x and y are in the same units)
`coord_quickmap()`	Approximately correct aspect for lat/long maps
`coord_polar()`	Convert to polar coordinates (bars → pie/Coxcomb charts)

> 💡 Tip
>
> Pie charts are bar charts in polar coordinates. The reason pie charts are usually worse than bars is exactly because polar coordinates make length comparisons harder than the original Cartesian bar.

Long labels: swap x and y

```r
ggplot(penguins, aes(y = species, x = body_mass_g)) +
  geom_boxplot(na.rm = TRUE) +
  labs(x = "Body mass (g)", y = NULL)
```



> **Note**
>
> Modern ggplot2 lets you just swap the `x =` and `y =` mappings directly (which is what we did above). The older approach was `+ coord_flip()` after defining x = species. Both work; the new way is cleaner.

The big picture

Where we are

You now have *every* layer of the ggplot2 template:

```r
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(
    mapping = aes(<MAPPINGS>),
    stat = <STAT>,
    position = <POSITION>
  ) +
  <COORDINATE_FUNCTION> +
  <FACET_FUNCTION> +
```

```
  <SCALE_FUNCTION> +
  labs(...)
```

The hard part isn't the syntax — it's deciding **what to put in each slot** to answer the question you actually have.

What we covered today

- **Multivariable plots:** color + shape + facet for 3–5 variables; faceting beats piling on aesthetics
- **Facets:** `facet_wrap` vs `facet_grid`, the `.` placeholder, free vs fixed scales, when to put a variable on rows vs columns
- **Specifying a graph:** frame → layer → refine
- **Axes & labels:** `labs()` is non-negotiable; `xlim` drops data, `coord_cartesian` doesn't; log scales when data spans orders of magnitude
- **Color scales:** viridis for continuous, qualitative palettes for categorical, diverging *only* when there's a meaningful midpoint
- **Coordinate systems:** flip, fixed, polar — useful for a few specific cases

Activity

Activity: plot-matching with penguins

Roughly 25 minutes. Working individually or with a neighbor.

- Activity Canvas Link

Format: I'll give you four target plots showing different combinations of penguin variables. Your job is to **recreate each one** as closely as you can using ggplot2, then write one sentence per plot about what it reveals.

This is the highest-leverage exercise for cementing the layer-by-layer thinking: you have to look at a plot, identify its frame, identify its glyphs, identify each aesthetic mapping, and work out which functions to call.

To turn in: Save your `.qmd` and rendered HTML — we'll check at the end of class.

Wrap-up & looking ahead

Tomorrow (Friday): Quiz 2 (first 20 min) + a session on responsible AI use for coding, with a hands-on activity testing whether you can spot what AI gets wrong.

Upcoming Deadlines

- **Fri 5/29, in class**: Quiz 2
- **Fri 5/29, 11:59 p.m.**: HW 2: Pipes, Plots & Reading Graphics

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

Syllabus · AI policy · Schedule · *R4DS* Ch. 9 · *DC* Ch. 8 · ggplot2 reference